4

Unit 4: Microprogrammed Control, CPU and Computer Arithmetic

Computer Organization and Architecture

CHANDRAS EDU

Never stop learning

Agenda

4.1 Control Memory
4.2 Address Sequencing
4.3 General Register Organization
4.4 Stack Organization
4.5 Instruction Formats
4.6 Addressing Modes
4.7 Data Transfer and Manipulation
4.8 Program Control
4.9 Addition and Subtraction
4.10 Multiplication Algorithms
4.11 Division Algorithms
4.12 Floating-Point Arithmetic Operations

4.1 Control Memory

IN SIMPLE TERMS

A microprogrammed control unit is a tiny computer inside the CPU. Its program (the microprogram) lives in a ROM called control memory. Each word it reads out (a microinstruction) switches on the control signals for one clock cycle.

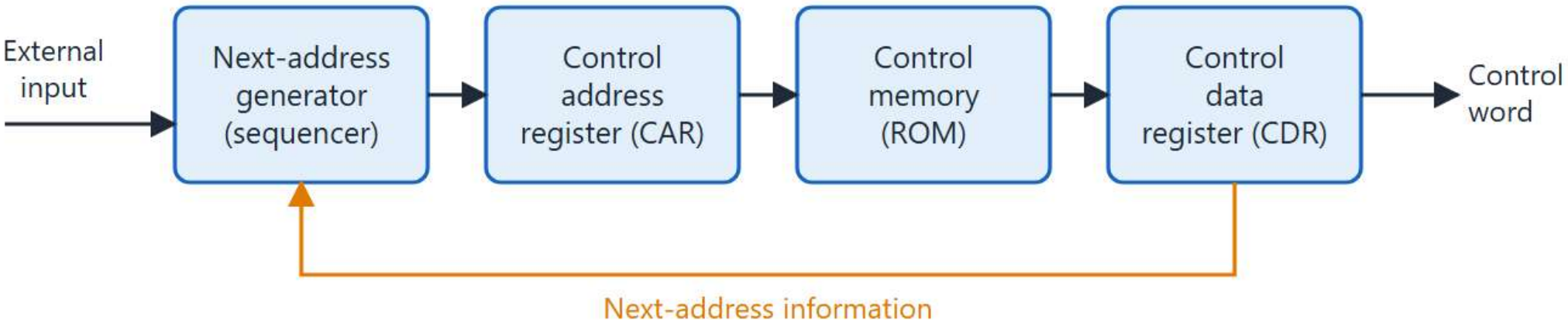
Microprogrammed control

- Every machine instruction triggers an ordered sequence of register transfers and ALU operations
- **Hardwired** control (logic gates alone) is fast but inflexible
- **Microprogrammed** control stores the control signals as binary patterns in the **control memory**
- The control unit reads them out one word at a time, like a processor reading instructions from main memory

Microinstructions and the microprogram

- Each word in control memory is a **microinstruction**: the microoperations (register transfers, ALU selections, memory-access signals) active in one clock cycle
- A sequence of microinstructions that implements one machine instruction is a **microprogram**
- The microprogram never changes, so control memory can be a **ROM**
- ROM is combinational: address in, word out; no clock edge is needed to read it
- **Dynamic microprogramming:** a writable control memory (RAM) lets a new microprogram be loaded from disk at boot, so one processor can emulate another

Microprogrammed control organization



Microprogrammed control organization. Adapted from M. Morris Mano, Computer System Architecture, Fig. 7-1.

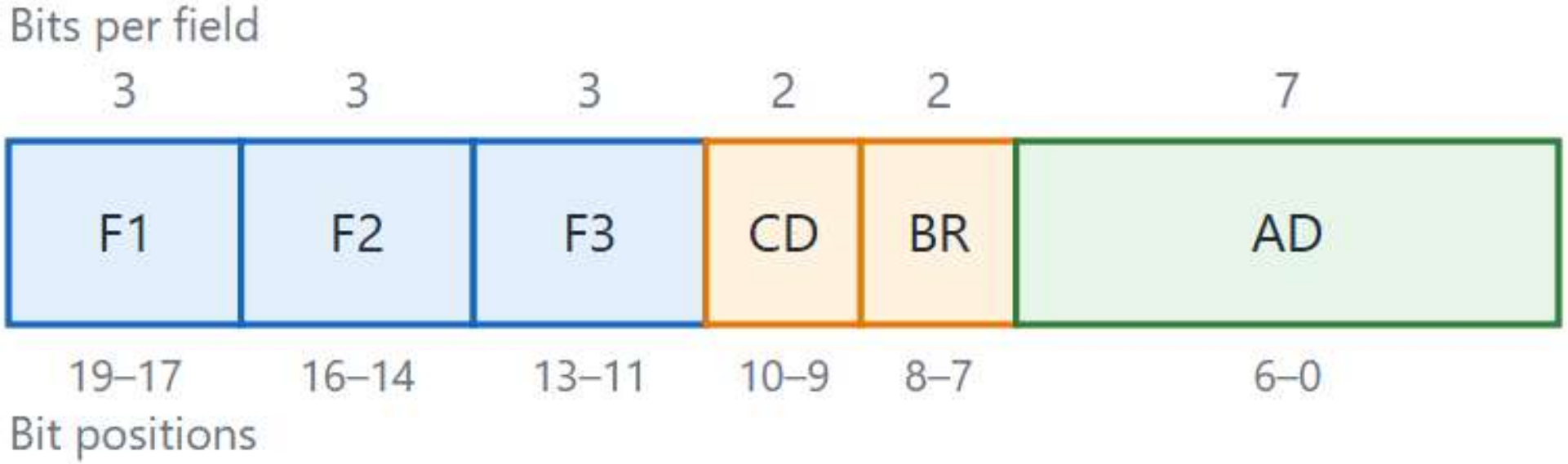
The four components

- **CAR** (control address register): the address of the current microinstruction
- **Sequencer:** computes the next address
- **ROM:** outputs the microinstruction at the CAR address
- **CDR** (pipeline register): holds the fetched microinstruction while the next address is computed
- Mano's Chapter 7 example omits the CDR and uses a single-phase clock, reading the control word straight from the ROM output

The microinstruction format

- Mano’s example control memory is 128 × 20: 128 microinstructions, each 20 bits wide, in six fields
- **F1, F2, F3** (nine bits): select up to three simultaneous microoperations; code 000 means no operation in that field
- **AD** (7 bits): the address field; $2^7 = 128$ matches the control memory size
- **CD:** selects which status bit to test
- **BR:** specifies the type of address transition

Microinstruction code format (20 bits)



F1, F2, F3: microoperation fields

CD: condition for branching

BR: branch field

AD: address field (7 bits → 128 words)

Microinstruction code format (20 bits). Adapted from M. Morris Mano, Computer System Architecture, Fig. 7-6.

4.2 Address Sequencing

IN SIMPLE TERMS

After each microinstruction the CAR must get a new address. It can step to the next one, jump to the AD address, jump to the start of the routine for the current machine instruction (mapping), or jump into and back from a subroutine.

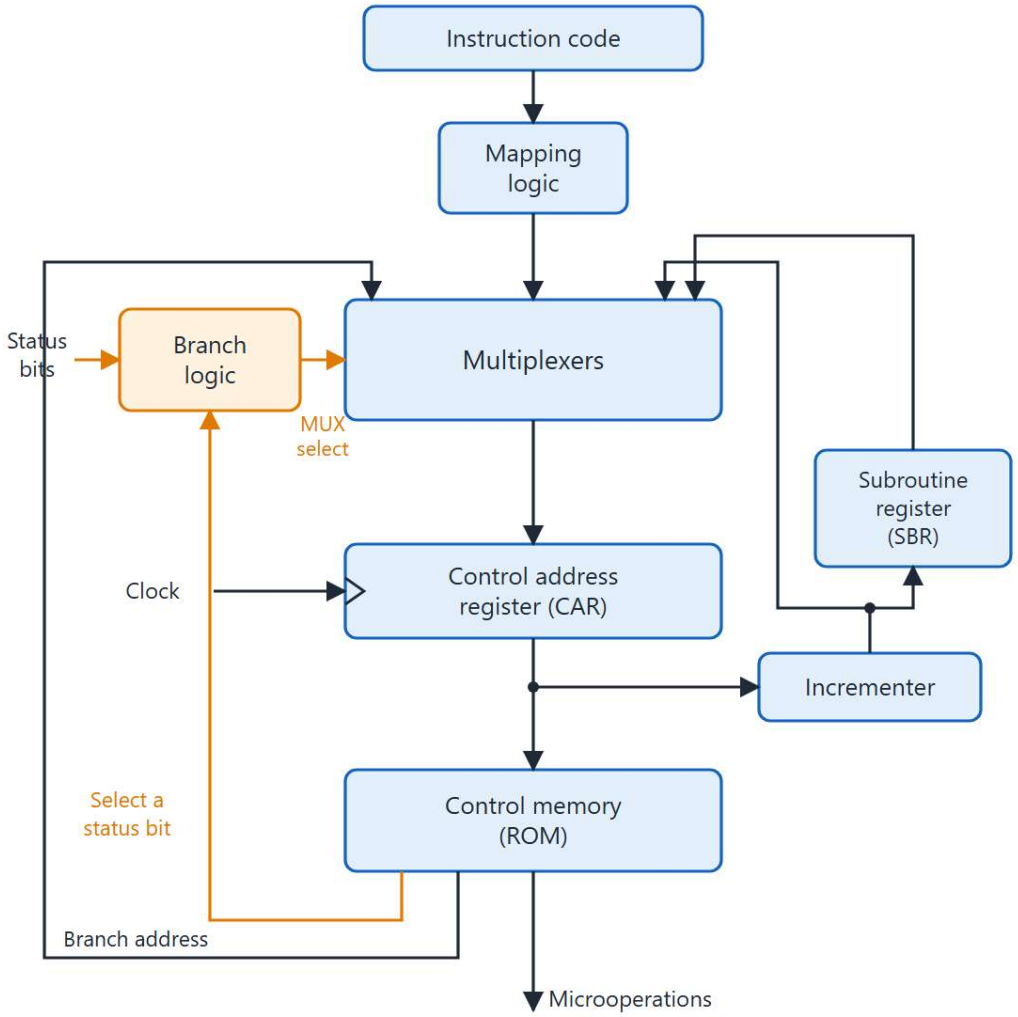
The microprogram sequencer

- A microprogram is not executed strictly from beginning to end
- Each machine instruction maps to its own microprogram routine
- The routines share a common fetch sequence
- Conditional branches may take different paths, depending on status bits
- The hardware that computes the next CAR address is the **microprogram sequencer**

Four required capabilities

- **1. Increment the CAR:** execute the next microinstruction in sequence
- **2. Branch:** jump to the AD address, always (unconditional) or only when a tested status bit has a given value (conditional)
- **3. Map:** at the start of each execute phase, jump to the routine for the current machine instruction (opcode → control memory address)
- **4. Subroutine call and return:** jump to a shared microprogram subroutine and return to the calling address

Selection of address for control memory



- The multiplexers choose the next address from four sources: CAR + 1, the AD field, the mapping logic and the SBR
- The incrementer output also feeds the SBR during a subroutine call
- The branch logic tests the status bit chosen by CD and drives the MUX selects
- The mapping logic turns the opcode into a starting address

Selection of address for control memory. Adapted from M. Morris Mano, Computer System Architecture, Fig. 7-2.

The BR field

BR	Symbol	Function
00	JMP	CAR ← AD if condition = 1; CAR ← CAR + 1 if condition = 0
01	CALL	CAR ← AD and SBR ← CAR + 1 if condition = 1; CAR ← CAR + 1 if condition = 0
10	RET	CAR ← SBR (return from subroutine)
11	MAP	CAR(2–5) ← DR(11–14), CAR(0, 1, 6) ← 0

Adapted from M. Morris Mano, Computer System Architecture, Table 7-1

The CD field and the mapping logic

- **CD** selects the condition: 00 = U (always 1), 01 = I (DR(15), the indirect bit), 10 = S (AC(15), the sign), 11 = Z (AC = 0)
- With CD = 00, JMP and CALL always take the branch
- The mapping logic is a PLD or a small ROM that turns opcode bits into a control memory address
- Nested subroutine calls need a hardware stack in the SBR, since one register saves only one return address

4.3 General Register Organization

IN SIMPLE TERMS

Pick two registers (SELA, SELB), pick an operation (OPR), pick where the answer goes (SELD). Those four choices, 14 bits in total, make one control word.

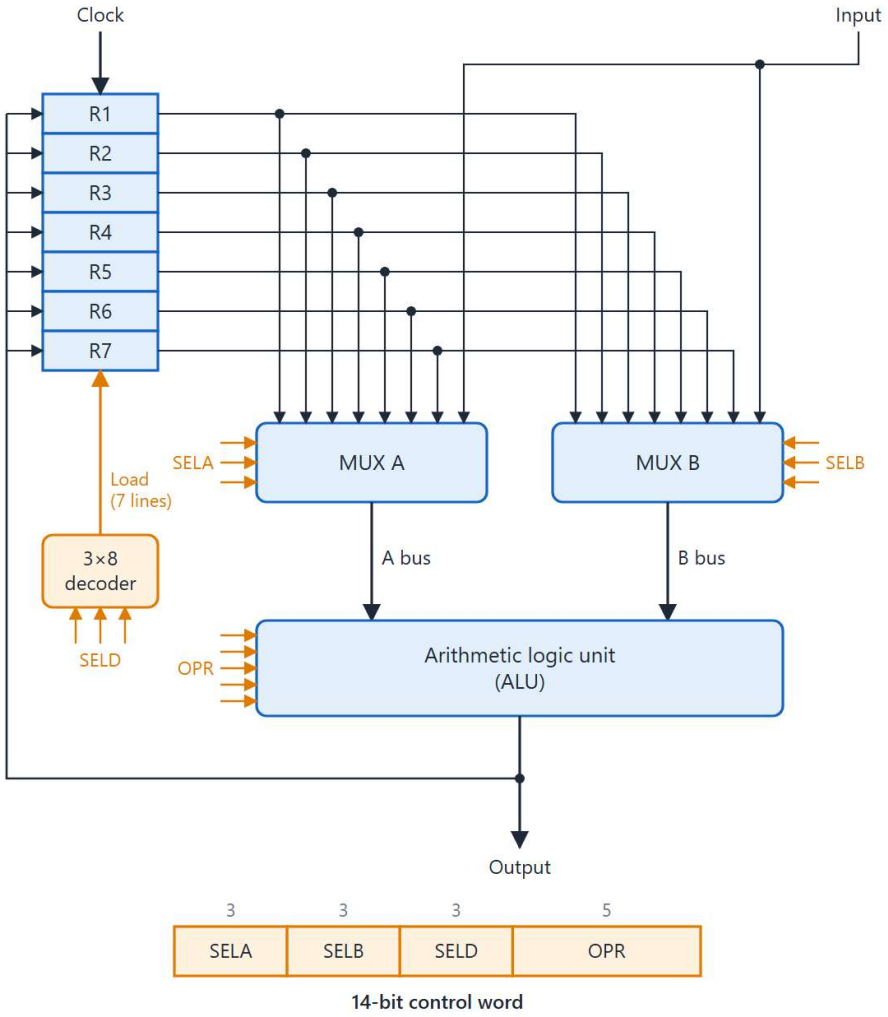
Why a register file

- Processors need fast temporary storage: partial products, pointer values, return addresses
- Main memory is too slow for this: a memory access is the slowest operation a CPU performs
- Solution: a small register file inside the CPU, connected through a shared bus and a single ALU
- This **general register organisation** is the internal datapath most modern CPUs are built around

The bus-organised register file

- Seven registers R1–R7 plus an external input path
- Every register output goes to two multiplexers
- The single ALU output goes back to all seven registers and to the external output
- A 3×8 decoder decides which one register loads the result
- Sharing one ALU needs far less wiring than connecting every register to every other

Register set with common ALU



- **MUX A** (SELA) and **MUX B** (SELB) pick the source registers for buses A and B
- The **ALU** performs the operation chosen by OPR
- The **3×8 decoder** (SELD) loads exactly one destination register
- Code 000 means external Input (SELA, SELB) or no destination (SELD)

Register set with common ALU (block diagram and 14-bit control word). Adapted from M. Morris Mano, Computer System Architecture, Fig. 8-2.

The 14-bit control word

- Four selector fields: SELA (3 bits) + SELB (3 bits) + SELD (3 bits) + OPR (5 bits) = 14 bits
- Code 000 selects the external Input (SELA/SELB) or no destination (SELD)
- Codes 001–111 select R1–R7
- There are eleven named ALU operations; some of them are listed on the next slide

Some ALU operation codes

OPR	Operation
00010	ADD
00101	SUB
01000	AND
01010	OR
01100	XOR
01110	COMA
10000	SHRA
11000	SHLA

Worked example: R1 ← R2 + R3


```
SELA = 010    (R2 on bus A)
SELB = 011    (R3 on bus B)
OPR  = 00010  (ADD)
SELD = 001    (result into R1)
Control word:  010 011 001 00010
```

- On the clock edge the ALU result is loaded into R1

4.4 Stack Organization

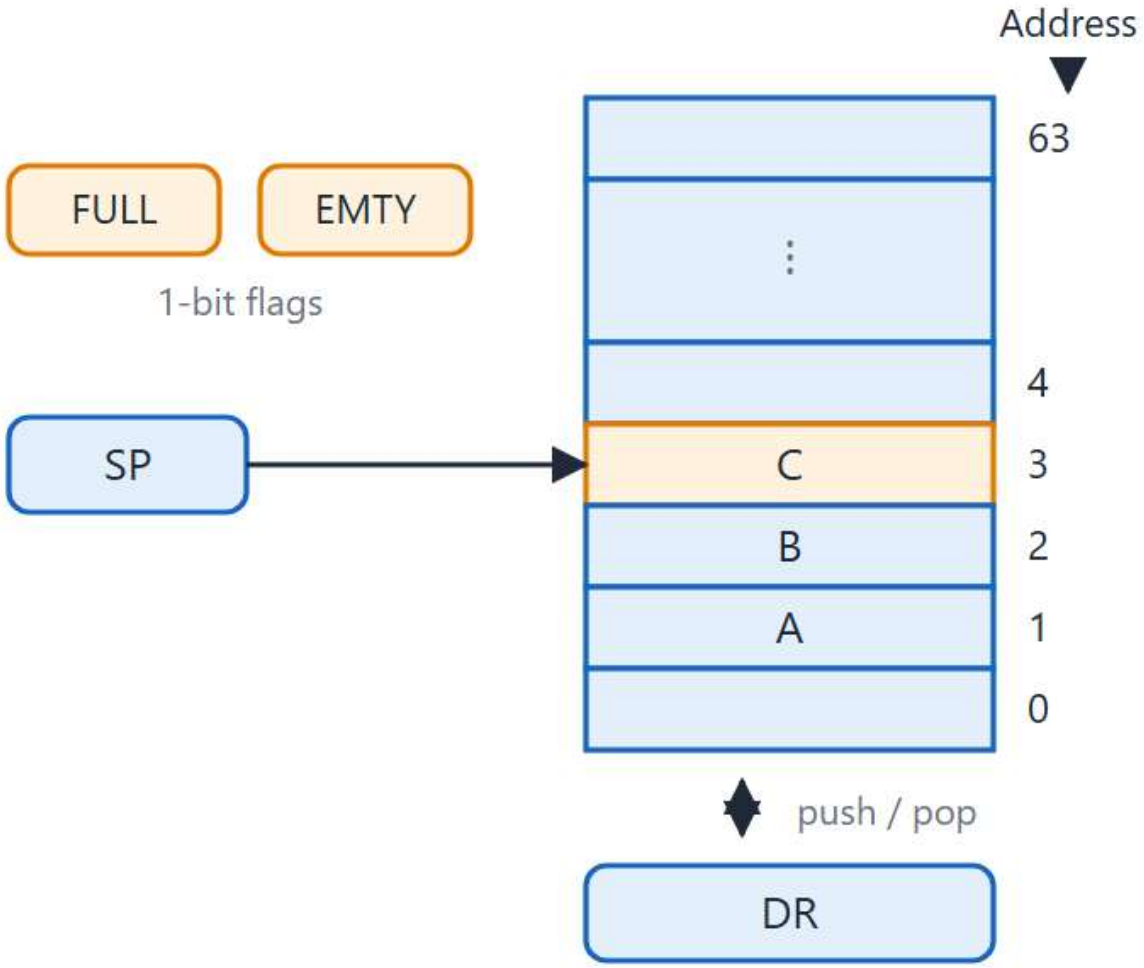
IN SIMPLE TERMS

A stack works like a pile of plates: the last item put on top is the first one taken off. The stack pointer (SP) always holds the address of the top item; push adds an item, pop removes one.

Stack basics and the register stack

- A stack is **last-in, first-out (LIFO)**: the most recently stored item is retrieved first
- It is a memory unit paired with a **stack pointer (SP)** that points to the top item
- **Register stack:** a 64-word unit; SP is a 6-bit address ($2^6 = 64$)
- **FULL** = 1 when the stack holds 64 items; **EMTY** = 1 when it is empty (Mano spells it EMTY)
- The data register **DR** is the channel through which all words enter and leave

Register stack



- SP (6 bits) points to the top item
- FULL = 1 when the stack holds 64 items; EMTY = 1 when it is empty
- DR is the channel for every word entering or leaving
- Initially SP = 0, EMTY = 1, FULL = 0; the first item goes to address 1

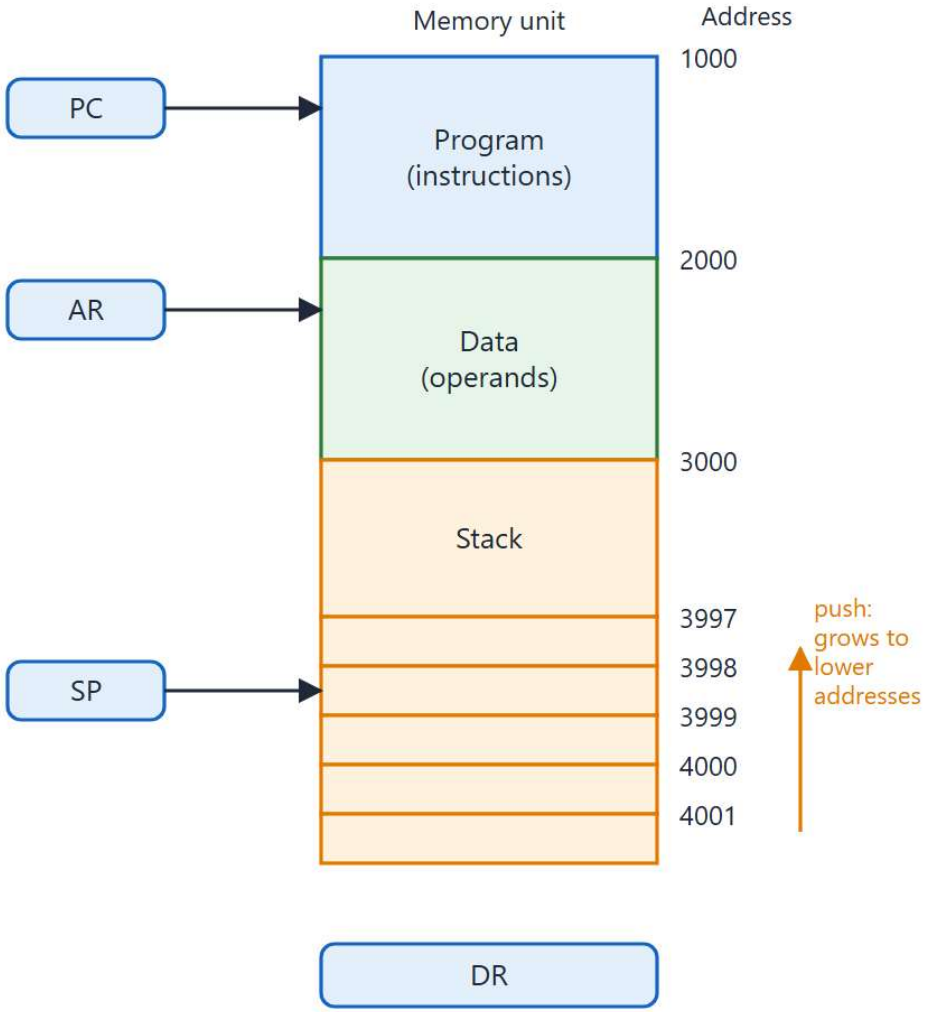
Adapted from M. Morris Mano, Computer System Architecture, Fig. 8-3

Push and pop RTL


```
Push (when FULL = 0):
SP ← SP + 1                      Increment stack pointer
M[SP] ← DR                       Write item on top of stack
If (SP = 0) then (FULL ← 1)      Check if stack is full
EMTY ← 0                         Mark stack not empty

Pop (when EMTY = 0):
DR ← M[SP]                       Read item from top of stack
SP ← SP - 1                      Decrement stack pointer
If (SP = 0) then (EMTY ← 1)      Check if stack is empty
FULL ← 0                         Mark stack not full
```

Memory stack



Adapted from M. Morris Mano, Computer System Architecture, Fig. 8-4

- A segment of main memory; SP is a processor register; PC, AR and SP supply addresses
- SP starts at 4001; the stack grows by decreasing address; lowest usable address is 3000
- Push: decrement SP, then write M[SP]. Pop: read M[SP], then increment SP
- Software checks the limits using two registers for the bounds

Reverse Polish notation

- Infix puts the operator between operands (A + B); **reverse Polish notation (RPN)**, also called postfix, puts it after: AB+
- RPN suits a stack: scan left to right and push each operand
- At an operator, pop the top two operands, apply it and push the result
- The final result stays on top of the stack
- Compilers routinely convert infix to RPN: it is the most efficient way to translate arithmetic into machine instructions

RPN evaluation: (3 × 4) + (5 × 6)

In RPN the expression is: 3 4 * 5 6 * +

Symbol	Stack (bottom → top)	Action
3	3	Push 3
4	3, 4	Push 4
*	12	Pop 4 and 3; push 3×4 = 12
5	12, 5	Push 5
6	12, 5, 6	Push 6
*	12, 30	Pop 6 and 5; push 5×6 = 30
+	42	Pop 30 and 12; push 12+30 = 42

4.5 Instruction Formats

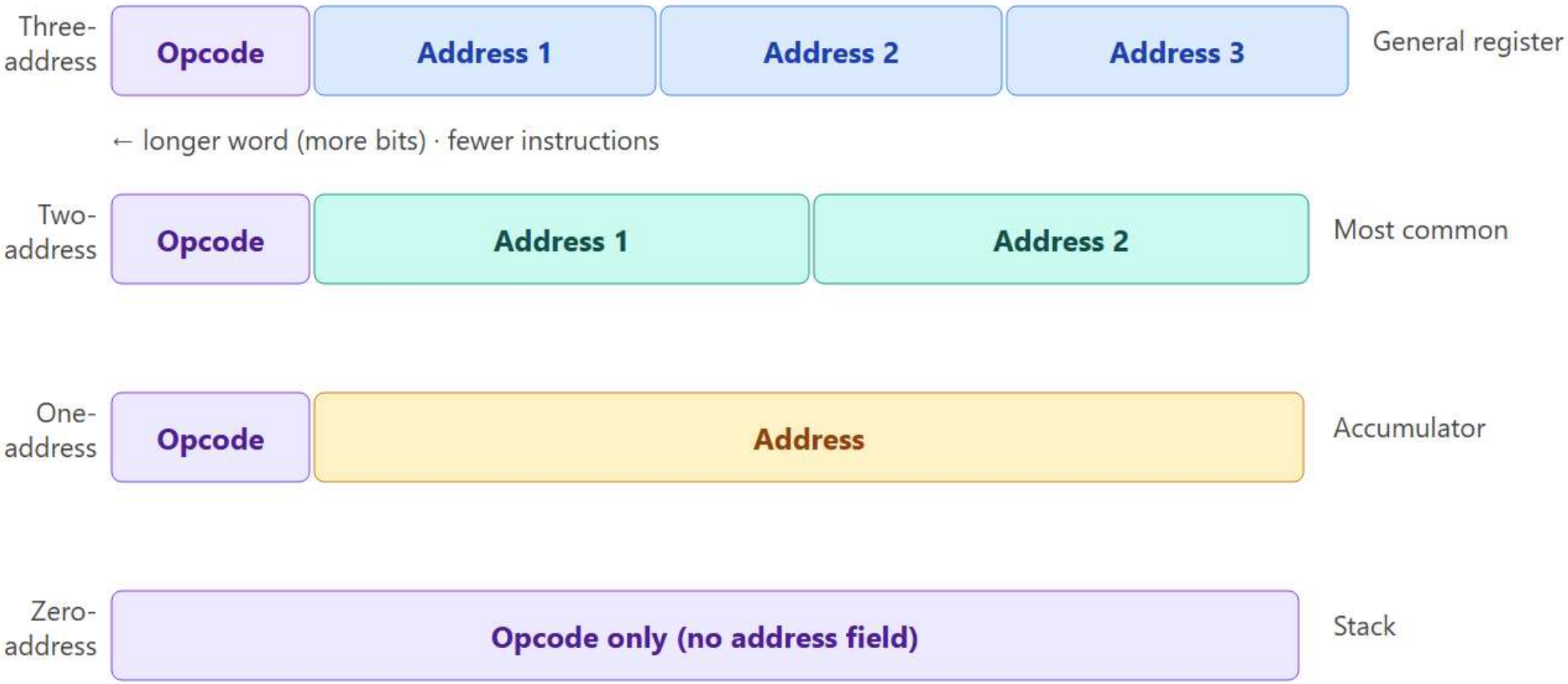
IN SIMPLE TERMS

An instruction is a row of bits split into fields: what to do (opcode), where the data is (address) and how to find it (mode). Fewer address fields make each instruction shorter but the program longer.

Fields in an instruction

- An instruction format is the binary layout of a machine instruction, divided into fields
- **Opcode field:** specifies the operation
- **Address field:** a memory address or a processor register; a k-bit register address selects one of 2^k registers (16 registers need 4 bits)
- **Mode field:** how the operand or effective address is determined
- Special fields exist, e.g. a shift-count field in a shift instruction
- The number of address fields sets the CPU organisation: single accumulator, general register or stack

Instruction format types



Adapted from M. Morris Mano, Computer System Architecture, §8-4

X = (A+B)*(C+D): three-address


```
ADD R1, A, B      R1 ← M[A] + M[B]
ADD R2, C, D      R2 ← M[C] + M[D]
MUL X, R1, R2     M[X] ← R1 * R2
```

- Operands are in memory at A, B, C, D; the result is stored at X
- Three instructions in total

X = (A+B)*(C+D): two-address


```
MOV R1, A       R1 ← M[A]
ADD R1, B       R1 ← R1 + M[B]
MOV R2, C       R2 ← M[C]
ADD R2, D       R2 ← R2 + M[D]
MUL R1, R2      R1 ← R1 * R2
MOV X, R1       M[X] ← R1
```

- MOV is needed to set up the registers

X = (A+B)*(C+D): one-address


```
LOAD A      AC ← M[A]
ADD B       AC ← AC + M[B]
STORE T     M[T] ← AC
LOAD C      AC ← M[C]
ADD D       AC ← AC + M[D]
MUL T       AC ← AC * M[T]
STORE X     M[X] ← AC
```

- All operations go through the implied accumulator AC

X = (A+B)*(C+D): zero-address


```
PUSH A      TOS ← A
PUSH B      TOS ← B
ADD         TOS ← (A + B)
PUSH C      TOS ← C
PUSH D      TOS ← D
ADD         TOS ← (C + D)
MUL         TOS ← (C+D)*(A+B)
POP X       M[X] ← TOS
```

- Stack computer; TOS = top of stack

4.6 Addressing Modes

IN SIMPLE TERMS

An addressing mode is the rule the CPU uses to find the data. The operand may be inside the instruction, in a register, in memory, or at an address the CPU must first calculate.

The idea of an addressing mode

- The mode gives the rule for interpreting or modifying the address field before the operand is referenced
- Two purposes: programmer flexibility (pointers, loop counters, array indexing, relocation) and fewer address bits
- The **effective address** is the final memory address computed by the mode
- The mode may be a separate field next to the opcode, or folded into one code with the opcode
- The control unit reads the mode during decode to locate the operand
- Two modes need no address field (implied, immediate); the other seven interpret it differently

Instruction format with mode field



Adapted from M. Morris Mano, Computer System Architecture, Fig. 8-6

Modes 1–3

- **1. Implied:** the operand is implicit in the instruction definition; no address field. Accumulator instructions and zero-address stack instructions are implied-mode
- **2. Immediate:** the operand field holds the actual operand, not its address; no memory access. The effective address is the address of the instruction's second word (201 in the example)
- **3. Register:** the address field names a processor register that holds the operand; no memory access; a k-bit field selects one of 2^k registers

Modes 4–6

- **4. Register indirect:** the register's contents give the memory address of the operand; fewer bits than a direct address; the programmer loads the address into the register first
- **5. Autoincrement or autodecrement:** like register indirect, but the register is incremented after (autoincrement) or decremented before (autodecrement) the access; good for stepping through a table or implementing a stack
- **6. Direct address:** the address field is the effective address; one access fetches the operand; in branches it is the branch target

Modes 7–9

- **7. Indirect address:** the address field points to a memory word that holds the effective address; two memory accesses beyond the instruction fetch
- **8. Relative address:** EA = address field (a signed displacement) + PC, already incremented past the instruction; gives position-independent code with few address bits
- **9. Indexed:** EA = address field + index register XR; the field holds the array base and XR the element offset; XR can be incremented to reach successive elements
- **Base register** mode is computed like indexed, but the register is used for program relocation; it is not a separate row in the table

Numerical example of addressing modes

Registers

PC = 200

R1 = 400

XR = 100

AC

Address	Memory
200	Load to AC | Mode
201	Address = 500
202	Next instruction
399	450
400	700
500	800
600	900
702	325
800	300

Orange rows: the two-word instruction and the next one

Adapted from M. Morris Mano, Computer System Architecture, Fig. 8-7

- Instruction at address 200: “Load to AC”, address part = 500
- After fetch: PC = 202, R1 = 400, XR = 100
- M[500] = 800, M[800] = 300, M[400] = 700
- M[702] = 325, M[600] = 900, M[399] = 450

Effective address and operand for the nine modes

Mode	Effective address	Operand (AC)	How computed
Direct address	500	800	EA = address part = 500
Immediate operand	201	500	Operand = address part itself; EA = 201
Indirect address	800	300	EA = M[500] = 800; operand = M[800] = 300
Relative address	702	325	EA = PC + 500 = 202 + 500 = 702
Indexed address	600	900	EA = XR + 500 = 100 + 500 = 600
Register	—	400	Operand = R1 = 400; no memory EA
Register indirect	400	700	EA = R1 = 400; operand = M[400] = 700
Autoincrement	400	700	EA = R1 = 400; R1 incremented to 401 after access
Autodecrement	399	450	R1 decremented to 399 before access; EA = 399

Adapted from M. Morris Mano, Computer System Architecture, Table 8-4

4.7 Data Transfer and Manipulation

IN SIMPLE TERMS

Most instructions either move data (load, store, move) or change it (add, AND, shift). This topic lists the typical instructions in each group.

Data transfer instructions

8 instructions. No arithmetic is performed: only the location of the data changes.

Name	Mnemonic	Typical action
Load	LD	Memory → processor register (usually the accumulator)
Store	ST	Processor register → memory
Move	MOV	Register → register (or memory ↔ memory on some architectures)
Exchange	XCH	Swap two registers, or a register and a memory word
Input	IN	I/O terminal → processor register
Output	OUT	Processor register → I/O terminal
Push	PUSH	Processor register → top of memory stack
Pop	POP	Top of memory stack → processor register

Typical data transfer instructions. Adapted from M. Morris Mano, Computer System Architecture, Table 8-5.

Arithmetic instructions

9 instructions. Multiply and divide may be absent on small processors.

Name	Mnemonic
Increment	INC
Decrement	DEC
Add	ADD
Subtract	SUB
Multiply	MUL
Divide	DIV
Add with carry	ADDC
Subtract with borrow	SUBB
Negate (2’s complement)	NEG

Typical arithmetic instructions. Adapted from M. Morris Mano, Computer System Architecture, Table 8-7.

Logical and bit manipulation instructions (1 of 2)

10 instructions in total. AND clears selected bits, OR sets them, XOR complements them.

Name	Mnemonic	Effect
Clear	CLR	All bits of operand → 0
Complement	COM	1's complement of operand
AND	AND	Bit-by-bit AND: used to clear selected bits (mask)
OR	OR	Bit-by-bit OR: used to set selected bits
Exclusive-OR	XOR	Bit-by-bit XOR: used to complement selected bits

Typical logical and bit manipulation instructions. Adapted from M. Morris Mano, Computer System Architecture, Table 8-8.

Logical and bit manipulation instructions (2 of 2)

Name	Mnemonic	Effect
Clear carry	CLRC	Carry flag → 0
Set carry	SETC	Carry flag → 1
Complement carry	COMC	Carry flag ← NOT carry
Enable interrupt	EI	Interrupt flip-flop → 1
Disable interrupt	DI	Interrupt flip-flop → 0

Typical logical and bit manipulation instructions. Adapted from M. Morris Mano, Computer System Architecture, Table 8-8.

Shift instructions

8 instructions covering 4 types: logical, arithmetic, rotate and rotate-through-carry; each has a left and a right variant.

Name	Mnemonic	Bit shifted in at the vacated end
Logical shift right	SHR	0
Logical shift left	SHL	0
Arithmetic shift right	SHRA	Sign bit (leftmost bit preserved)
Arithmetic shift left	SHLA	0 (same as logical shift left)
Rotate right	ROR	Rightmost bit wraps to leftmost
Rotate left	ROL	Leftmost bit wraps to rightmost
Rotate right through carry	RORC	Carry bit; rightmost bit → carry
Rotate left through carry	ROLC	Carry bit; leftmost bit → carry

Typical shift instructions. Adapted from M. Morris Mano, Computer System Architecture, Table 8-9.

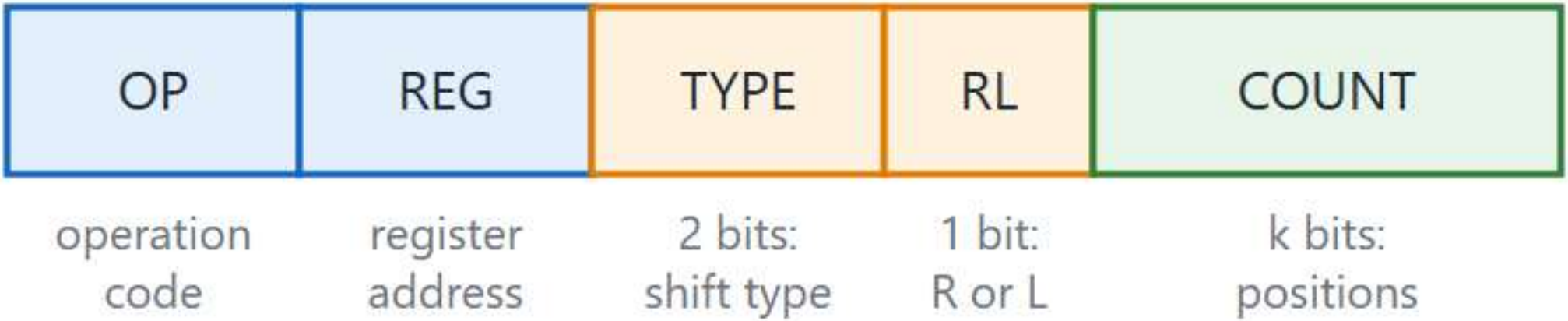
Shift types

- **Logical shift:** inserts 0 at the vacated end; useful for unsigned multiplication or division by powers of 2
- **Arithmetic shift right:** preserves the sign bit, keeping the sign correct in 2's complement arithmetic
- **Arithmetic shift left:** inserts 0; identical to logical shift left
- **Rotate:** circular; no bit is lost, the bit shifted out reappears at the other end
- **Rotate through carry:** the carry flag acts as an extra bit, giving a 9-bit (17-bit on a 16-bit machine) circular shift

Shift instruction format

- Some processors encode all shift information in one multi-field instruction
- Five fields: **OP, REG, TYPE** (2 bits), **RL** (1 bit), **COUNT** (k bits)
- TYPE (2 bits) encodes the four shift variants
- RL (1 bit) selects the direction
- COUNT allows up to $2^k - 1$ positions in one instruction
- One instruction specifies the type, direction and count together

Shift instruction format



TYPE: logical, arithmetic, rotate, rotate-through-carry

RL: 0 = right, 1 = left · COUNT: up to $2^k - 1$ positions

Shift instruction format. Adapted from M. Morris Mano, Computer System Architecture, §8-6.

4.8 Program Control

IN SIMPLE TERMS

The CPU normally runs instructions one after another. Program control instructions change that order by branching, calling a subroutine or handling an interrupt, often based on status bits set by the last ALU operation.

Changing the program flow

- Normally the PC is incremented after every fetch, so instructions run in memory order
- Program control instructions load a new value into the PC, so the CPU branches elsewhere
- They cover branches, calls, returns and interrupts
- Compare and test instructions update status bits; a later conditional branch then tests them

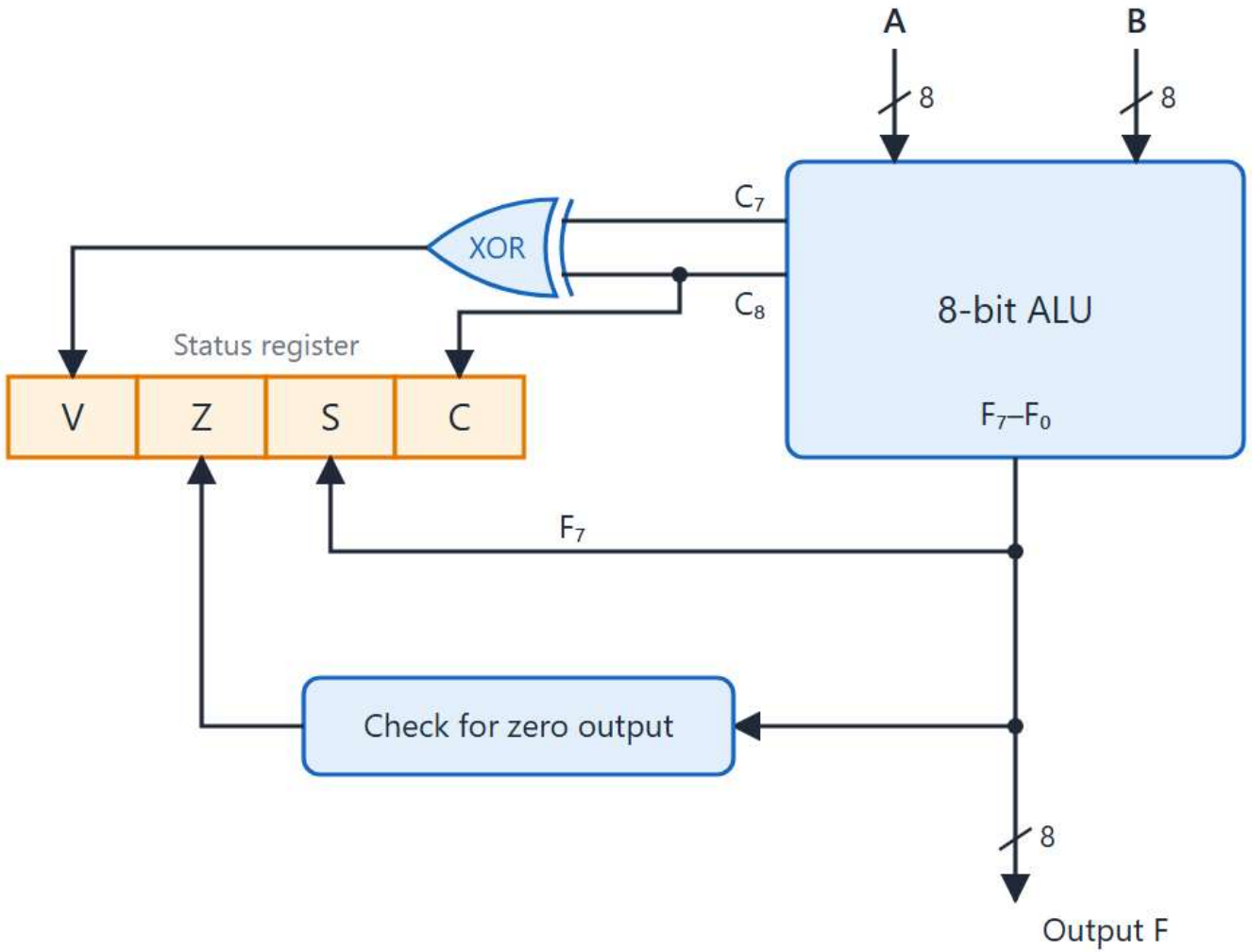
Program control instructions

7 instructions. Branch and jump are used interchangeably in many architectures.

Name	Mnemonic	Action
Branch	BR	PC ← branch address (conditional or unconditional)
Jump	JMP	Unconditional transfer to specified address
Skip	SKP	Increment PC by 1 extra if condition is met (skip next instruction)
Call	CALL	Save return address; transfer to subroutine
Return	RET	Restore return address into PC; resume caller
Compare (by subtraction)	CMP	Set status bits from A − B; result discarded
Test (by ANDing)	TST	Set status bits from A AND B; result discarded

Typical program control instructions. Adapted from M. Morris Mano, Computer System Architecture, Table 8-10.

Status register bits



Status register bits. Adapted from M. Morris Mano, Computer System Architecture, Fig. 8-8.

The four status bits

- **C (carry):** 1 if the end carry C_8 is 1 after an addition; otherwise 0
- **S (sign):** 1 if the highest-order bit F_7 of the result is 1 (negative in 2's complement)
- **Z (zero):** 1 if all bits of the ALU output are 0
- **V (overflow):** 1 if $C_7 \oplus C_8 = 1$: 2's complement overflow
- For an 8-bit ALU, V = 1 means the result is greater than +127 or less than −128
- Overflow occurs exactly when the carry into the sign bit differs from the carry out of it

Conditional branch instructions (1 of 3)

Each mnemonic starts with B (branch), followed by an abbreviation of the condition.

Mnemonic	Branch condition	Tested condition
Flag-based conditions		
BZ	Branch if zero	Z = 1
BNZ	Branch if not zero	Z = 0
BC	Branch if carry	C = 1
BNC	Branch if no carry	C = 0
BP	Branch if plus	S = 0
BM	Branch if minus	S = 1
BV	Branch if overflow	V = 1
BNV	Branch if no overflow	V = 0

Conditional branch instructions. Adapted from M. Morris Mano, Computer System Architecture, Table 8-11.

Conditional branch instructions (2 of 3)

The words higher and lower apply to unsigned comparison.

Mnemonic	Branch condition	Tested condition
Unsigned compare conditions (A – B)		
BHI	Branch if higher	A > B
BHE	Branch if higher or equal	A ≥ B
BLO	Branch if lower	A < B
BLOE	Branch if lower or equal	A ≤ B
BE	Branch if equal	A = B
BNE	Branch if not equal	A ≠ B

Conditional branch instructions. Adapted from M. Morris Mano, Computer System Architecture, Table 8-11.

Conditional branch instructions (3 of 3)

Greater than and less than apply to signed comparison, where V and S must both be considered. BE and BNE appear in both groups: the equality test (Z = 1) is the same.

Mnemonic	Branch condition	Tested condition
Signed compare conditions (A – B)		
BGT	Branch if greater than	A > B
BGE	Branch if greater or equal	A ≥ B
BLT	Branch if less than	A < B
BLE	Branch if less or equal	A ≤ B
BE	Branch if equal	A = B
BNE	Branch if not equal	A ≠ B

Conditional branch instructions. Adapted from M. Morris Mano, Computer System Architecture, Table 8-11.

Subroutine call and return


```
CALL subroutine:
SP ← SP - 1                 Decrement stack pointer
M[SP] ← PC                  Push return address onto stack
PC ← effective address      Transfer control to subroutine

Return from subroutine (RET):
PC ← M[SP]      Pop return address into PC
SP ← SP + 1     Increment stack pointer
```

Program interrupt

- An interrupt transfers control to a service routine in response to an asynchronous event or an error
- **External:** from I/O devices, timing circuits or power-failure detectors; asynchronous
- **Internal (traps):** illegal or erroneous instructions, such as overflow, division by zero, invalid opcode, stack overflow, protection violation; synchronous
- **Software:** a special instruction (supervisor call), e.g. to switch from user mode to supervisor mode
- The CPU pushes the PC and the program status word (PSW) on the stack and loads the service routine's address into the PC
- Return from interrupt pops the PSW and PC, restoring the CPU state exactly

4.9 Addition and Subtraction

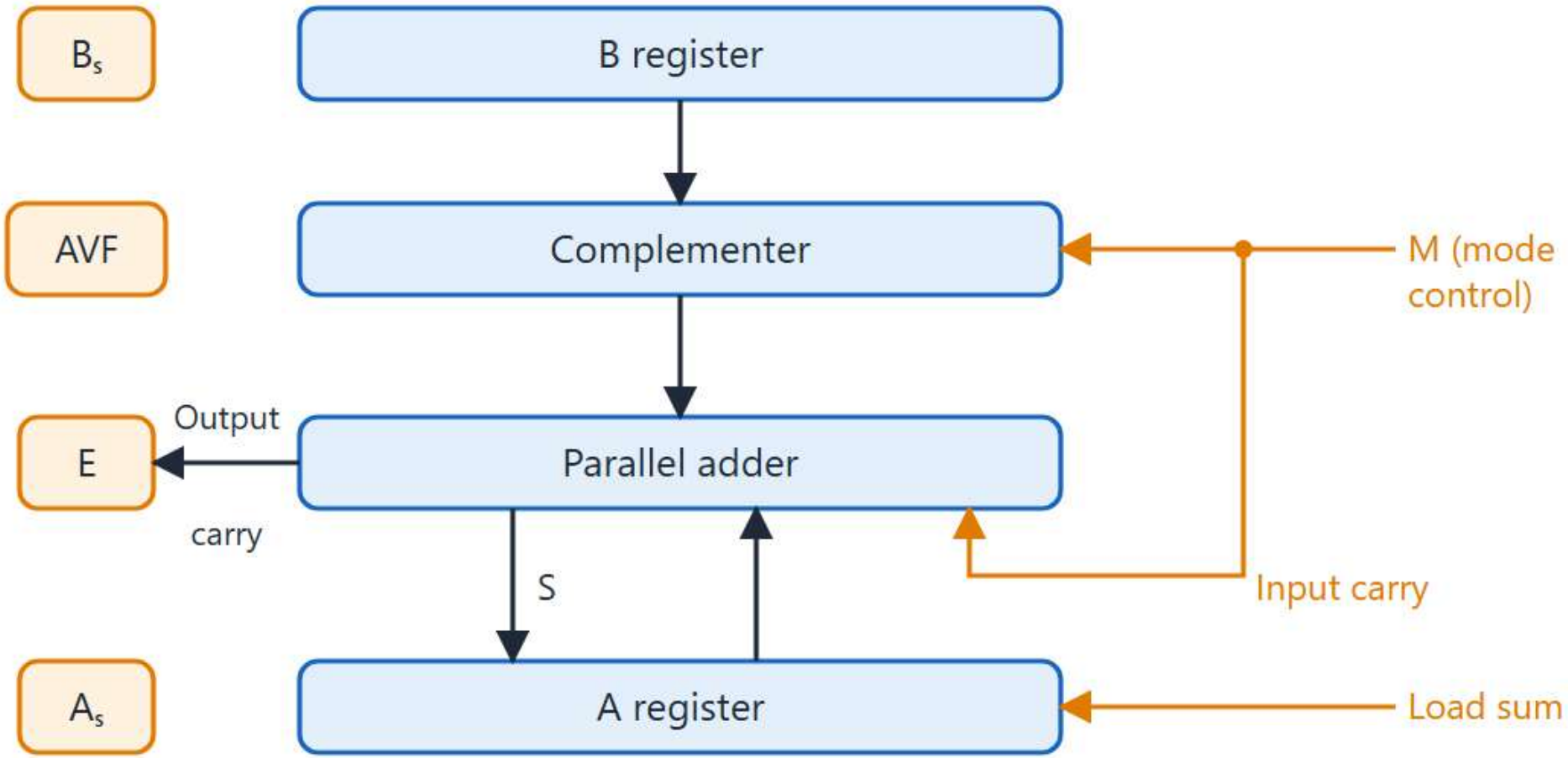
IN SIMPLE TERMS

With signs stored separately, the hardware first looks at the operation and the two signs, then either adds the magnitudes or subtracts them. Subtraction is done by adding the 2’s complement, so one adder handles both.

Signed-magnitude addition and subtraction

- A signed-magnitude number is a magnitude plus a separate sign bit
- The operation on the magnitudes depends on the operation and on both signs: 8 combinations
- **Add magnitudes** when the signs are identical (addition) or different (subtraction); the result takes the sign of A
- **Subtract magnitudes** otherwise: if A ≥ B the result has the sign of A; if A < B the result is B − A with the complement of A's sign
- A zero result must get a positive sign (+0, not −0)

Hardware for signed-magnitude addition and subtraction

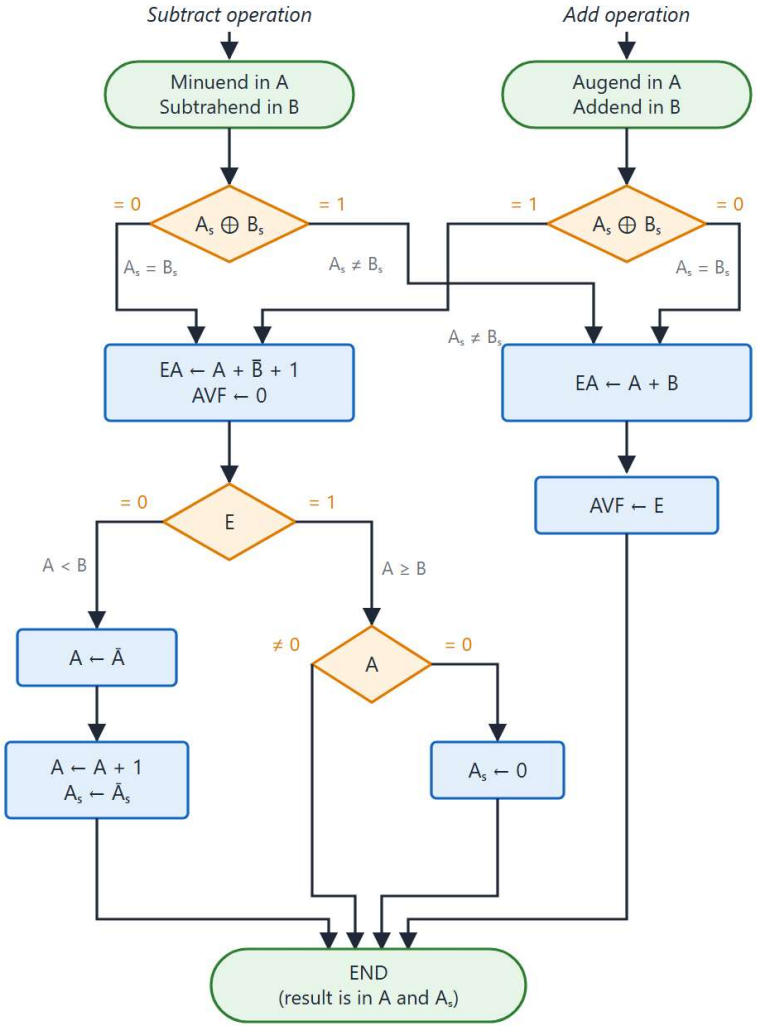


Hardware for signed-magnitude addition and subtraction. Adapted from M. Morris Mano, Computer System Architecture, Fig. 10-1.

Hardware components

- **B register:** holds the magnitude of the second operand
- **Complementer** (mode bit M): M = 0 passes B; M = 1 passes the 1's complement of B and sets the input carry to 1, so the adder computes A + B′ + 1 = A − B
- **Parallel adder:** computes the sum; its output carry goes to flip-flop E, showing A ≥ B after a subtraction
- **A register:** receives the result
- **Sign flip-flops** A_s and B_s: their XOR decides whether to add or subtract magnitudes
- **AVF** (add-overflow flip-flop): captures E when magnitudes are added; signals overflow

Add and subtract flowchart



- Compute $A_s \oplus B_s$: 0 means subtract magnitudes (EA ← A + B′ + 1, AVF ← 0); 1 means add magnitudes (EA ← A + B, AVF ← E)
- Subtract path, E = 1: A ≥ B; if A = 0 force $A_s \leftarrow 0$ to avoid −0
- Subtract path, E = 0: A < B; take the 2's complement of A and complement the sign
- Both columns share the same END node

Flowchart for add and subtract operations. Adapted from M. Morris Mano, Computer System Architecture, Fig. 10-2.

Signed-2’s complement arithmetic

- Most computers use signed-2’s complement; the algorithm is much simpler
- **Addition:** add the numbers including the sign bits; discard the carry out of the sign position
- **Subtraction:** take the 2’s complement of the subtrahend and add it to the minuend
- Same complementer-adder, but no separate sign flip-flops; the registers are AC and BR
- **Overflow rule:** adding two numbers of the same sign gives a result of the opposite sign
- $V = C_7 \oplus C_8$, where C_7 is the carry into the sign bit and C_8 the carry out of it

4.10 Multiplication Algorithms

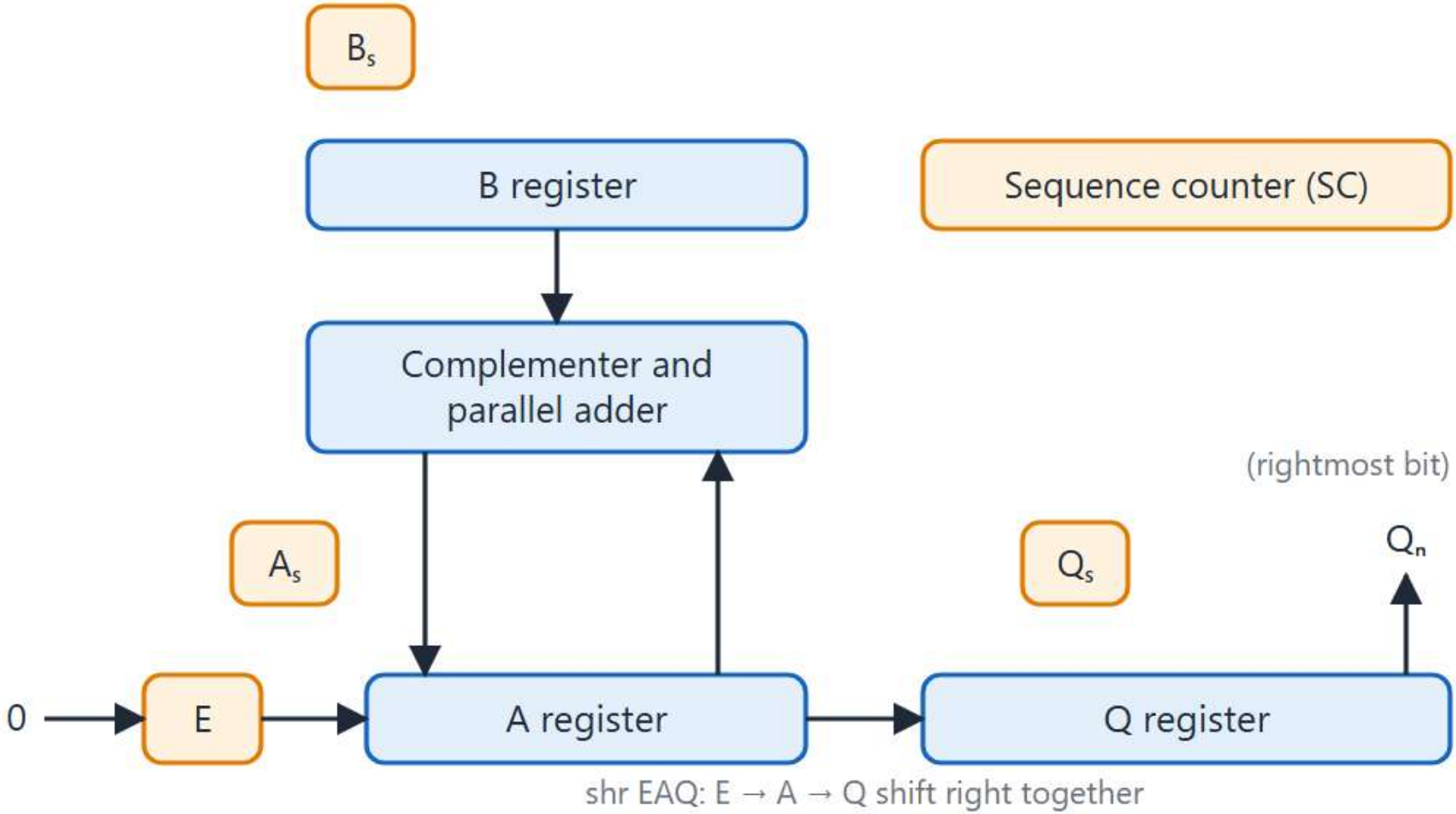
IN SIMPLE TERMS

The computer multiplies the way you do on paper: add the multiplicand when the multiplier bit is 1, then shift. But it shifts the partial product right instead of shifting the multiplicand left. Booth's method does the same for signed numbers by subtracting at the start of a run of 1s and adding at its end.

Signed-magnitude multiplication: hardware

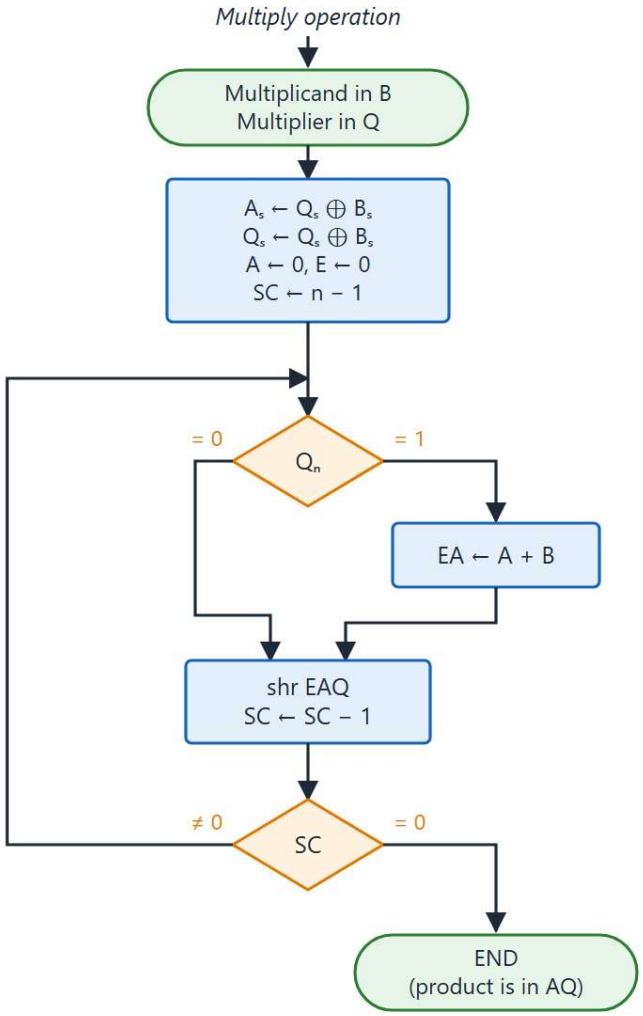
- Paper and pencil: add shifted copies of the multiplicand whenever the multiplier bit is 1
- Hardware: shift the **partial product** right after each addition, so everything stays in fixed-width registers
- **B:** multiplicand (magnitude); **A:** partial product, cleared to 0 initially
- **Q:** multiplier; its rightmost bit Q_n is tested each step
- **E:** carry extension left of A, cleared initially; **SC:** counts down from n − 1 to 0
- Sign flip-flops: the product sign is $A_s = Q_s \oplus B_s$

Hardware for multiply operation



Hardware for multiply operation. Adapted from M. Morris Mano, Computer System Architecture, Fig. 10-5.

Multiply flowchart



- If $Q_n = 1$ (rightmost bit of Q), EA ← A + B
- Then shr EAQ and SC ← SC − 1
- After n steps (SC = 0) the product is in AQ
- SC starts at n − 1: the first partial product is added before the first shift

Flowchart for multiply operation. Adapted from M. Morris Mano, Computer System Architecture, Fig. 10-6.

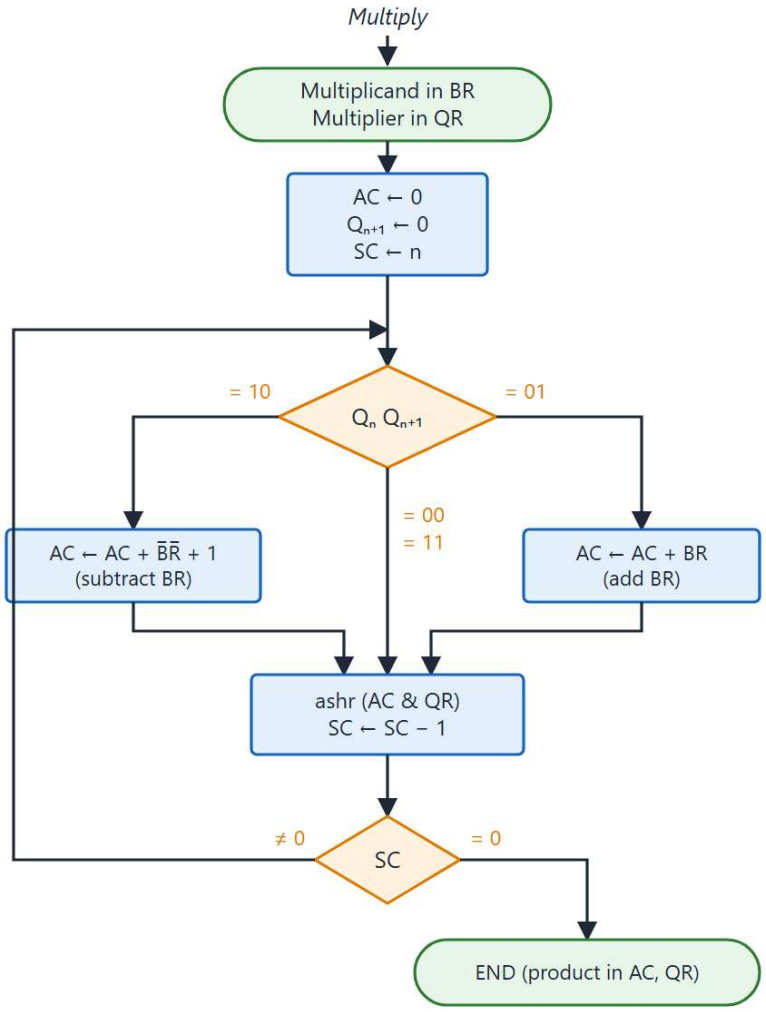
Booth’s algorithm: action for each bit pair

Booth multiplies signed 2’s complement numbers directly. A string of 1s is replaced by a subtraction at its low end and an addition at its high end. After any arithmetic, ashr is applied to AC and QR together, preserving the sign of AC.

Q_n	Q_{n+1}	Action	Meaning
1	0	AC ← AC + BR’ + 1 (subtract BR)	Beginning of a string of 1s
0	1	AC ← AC + BR (add BR)	End of a string of 1s
0	0	No arithmetic (shift only)	Middle of a string of 0s
1	1	No arithmetic (shift only)	Middle of a string of 1s

Booth algorithm: action for each bit pair

Booth algorithm flowchart



- Initialise: AC ← 0, Q_{n+1} ← 0, SC ← n
- Repeat n times: inspect $Q_n Q_{n+1}$ and do the action from the table
- Then ashr (AC & QR) and decrement SC
- When SC = 0 the product is in AC and QR

Booth algorithm for multiplication of signed-2's complement numbers. Adapted from M. Morris Mano, Computer System Architecture, Fig. 10-8.

Booth example: (−9) × (−13), n = 5 (1 of 2)

BR = 10111 (−9), QR = 10011 (−13), BR′ + 1 = 01001

$Q_n\ Q_{n+1}$	Operation	AC	QR	Q_{n+1}	SC
Initial		00000	10011	0	101
1 0	Subtract BR	01001			
	ashr	00100	11001	1	100
1 1	ashr	00010	01100	1	011
0 1	Add BR	10111 → 11001			
	ashr	11100	10110	0	010

Booth algorithm trace: BR = 10111 (−9), QR = 10011 (−13), product = +117. Adapted from M. Morris Mano, Computer System Architecture, Table 10-3.

Booth example: (–9) × (–13), n = 5 (2 of 2)

$Q_n\ Q_{n+1}$	Operation	AC	QR	Q_{n+1}	SC
0 0	ashr	11110	01011	0	001
1 0	Subtract BR	01001 → 00111			
	ashr	00011	10101	1	000
Product in AC·QR		**0001110101 = +117 ✓**			

Booth algorithm trace: BR = 10111 (−9), QR = 10011 (−13), product = +117. Adapted from M. Morris Mano, Computer System Architecture, Table 10-3.

Array multiplier

- The sequential algorithm needs n clock cycles
- A combinational **array multiplier** produces the product in a single step, using parallel hardware
- For j multiplier bits and k multiplicand bits it needs j × k AND gates and (j − 1) k-bit adders, producing a product of j + k bits
- Combinational (single step) but expensive in hardware

4.11 Division Algorithms

IN SIMPLE TERMS

The hardware divides like long division in binary. Each step shifts the partial remainder left and tries subtracting the divisor; if it fits, the quotient bit is 1, otherwise the remainder is restored and the bit is 0.

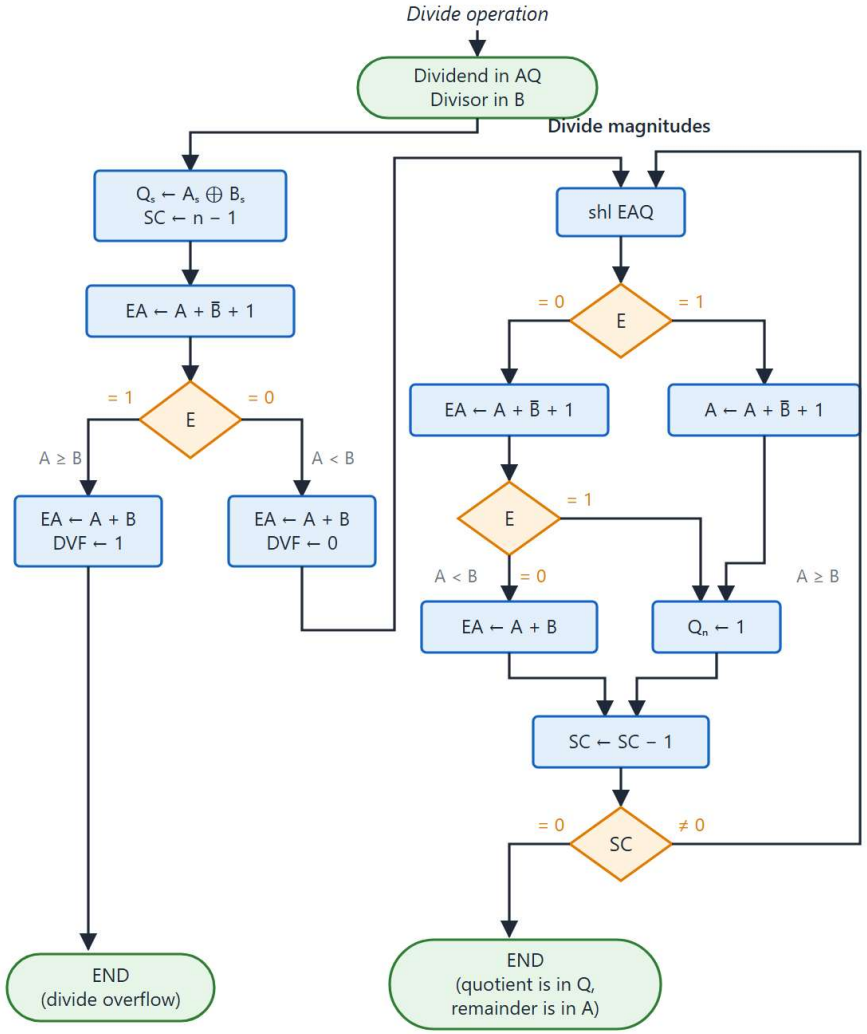
Division hardware: overview

- The hardware reuses the complementer-and-adder used for add, subtract and multiply
- The double-length dividend is in registers A and Q; the divisor is in B
- Each step: **1.** shl EAQ (shift the partial remainder left); **2.** EA ← A + B′ + 1 (trial subtraction)
- **3.** If E = 1 (A ≥ B), Q_n ← 1; if E = 0 (A < B), restore A by adding B back and the quotient bit stays 0
- **4.** Decrement SC
- After n steps the quotient is in Q and the remainder is in A

Divide overflow

- Overflow: the quotient needs more bits than the register holds
- It is detected **before** the main loop: if the high half of the dividend (initial A) is ≥ the divisor B, the quotient would exceed n bits
- Then the divide-overflow flip-flop **DVF** is set to 1 and the operation is aborted; hardware typically raises an interrupt
- **Condition:** DVF ← 1 when E = 1 after the first EA ← A + B′ + 1, i.e. A ≥ B before any shift
- Division by zero is also caught here, since any dividend is greater than zero

Divide flowchart (signed-magnitude)



- Initialise: $Q_s \leftarrow A_s \oplus B_s$, $SC \leftarrow n - 1$
- Overflow check: $EA \leftarrow A + B' + 1$; $E = 1$ sets $DVF \leftarrow 1$ and ends, $E = 0$ enters the loop
- Loop n times: shl EAQ, trial subtract; $E = 1$ sets $Q_n \leftarrow 1$, $E = 0$ restores A; $SC \leftarrow SC - 1$
- SC starts at n − 1: the overflow-check step counts as one

Flowchart for divide operation (signed-magnitude). Adapted from M. Morris Mano, Computer System Architecture, Fig. 10-13.

Division example: 448 ÷ 17 (1 of 2)

Dividing 0111000000 (448) by B = 10001 (17), following the steps of the book's worked example.

Step	Operation	E	A	Q	SC
Init	Dividend in AQ, divisor B = 10001, B′ + 1 = 01111	0	01110	00000	5
1	shl EAQ	0	11100	00000	
	Add B′ + 1 → E = 1: set Q_n = 1	1	01011	00001	4
2	shl EAQ	0	10110	00010	
	Add B′ + 1 → E = 1: set Q_n = 1	1	00101	00011	3

Binary division with digital hardware (Dividend = 01110 00000, Divisor B = 10001). Adapted from M. Morris Mano, Computer System Architecture, Fig. 10-12.

Division example: 448 ÷ 17 (2 of 2)

Quotient in Q = 11010 (26), remainder in A = 00110 (6): 448 ÷ 17 = 26 remainder 6.

Step	Operation	E	A	Q	SC
3	shl EAQ	0	01010	00110	
	Add B′ + 1 → E = 0: leave Q_n = 0, add B (restore)	1	01010	00110	2
4	shl EAQ	0	10100	01100	
	Add B′ + 1 → E = 1: set Q_n = 1	1	00011	01101	1
5	shl EAQ	0	00110	11010	
	Add B′ + 1 → E = 0: leave Q_n = 0, add B (restore)	1	00110	11010	0

Binary division with digital hardware (Dividend = 01110 00000, Divisor B = 10001). Adapted from M. Morris Mano, Computer System Architecture, Fig. 10-12.

4.12 Floating-Point Arithmetic Operations

IN SIMPLE TERMS

A floating-point number is a mantissa times a power of 2. To add, first make the exponents equal by shifting a mantissa; to multiply, add the exponents and multiply the mantissas; to divide, subtract the exponents and divide the mantissas. Then normalise.

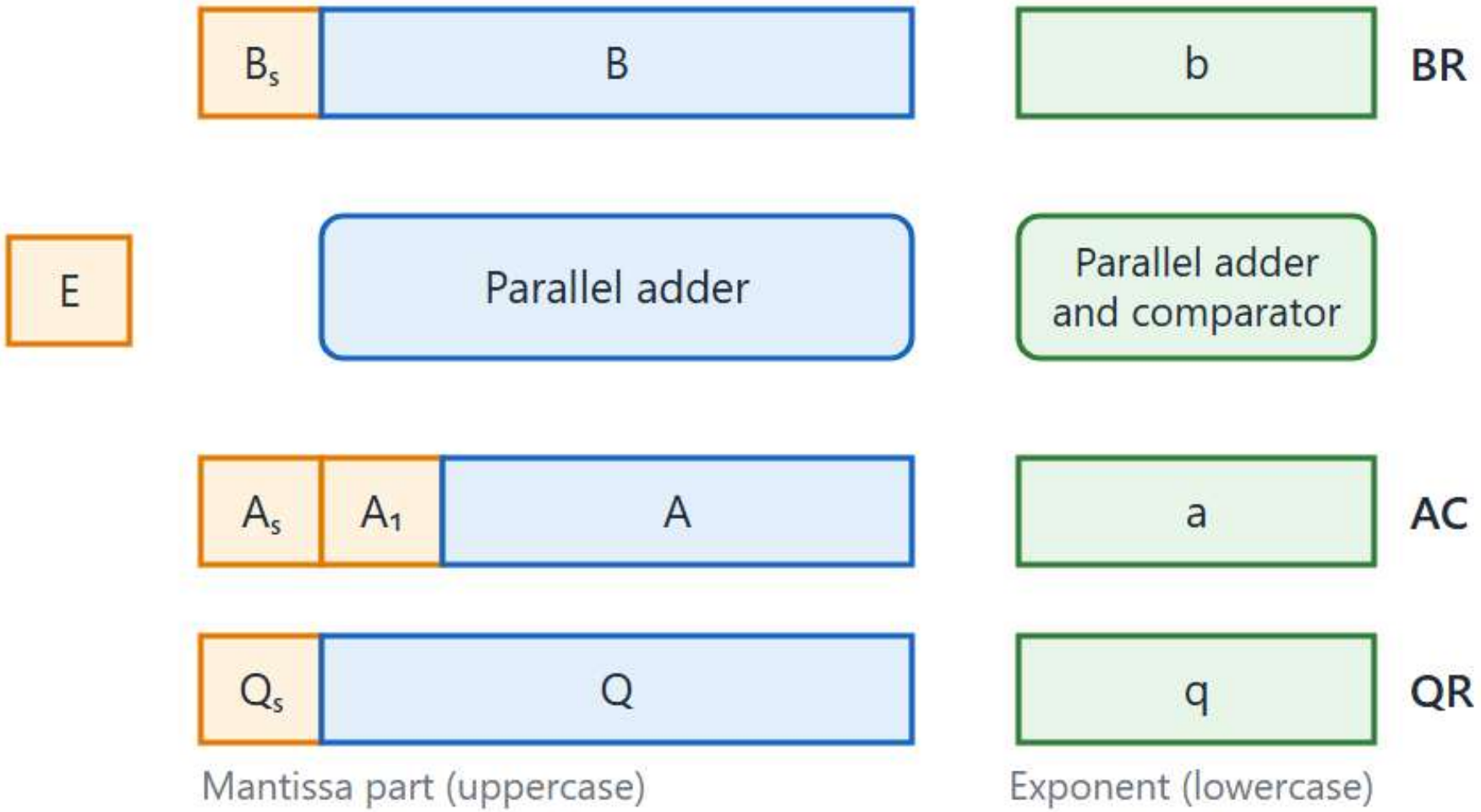
Floating-point basics

- A floating-point number has a **mantissa** m and an **exponent** e: the value is $m \times r^e$, with radix r
- The radix and the radix point are implied, not stored; here r = 2 and the mantissa is a fraction
- A number is **normalised** when the most significant digit of the mantissa is nonzero; this maximises the significant digits
- Zero cannot be normalised: it is all 0s in both the mantissa and exponent fields

Biased exponent

- The exponent can be stored as signed-magnitude, signed-2's complement, signed-1's complement or **biased**
- Biased is the most common: a positive bias is added to every exponent before storing
- Example: bias 50, exponent range −50 to +49, stored values 0 to 99: all positive, no sign bit
- **Advantage:** all stored exponents are positive, so a magnitude comparator can compare them; this simplifies the mantissa alignment hardware

Register configuration for floating-point operations



Register configuration for floating-point arithmetic operations. Adapted from M. Morris Mano, Computer System Architecture, Fig. 10-14.

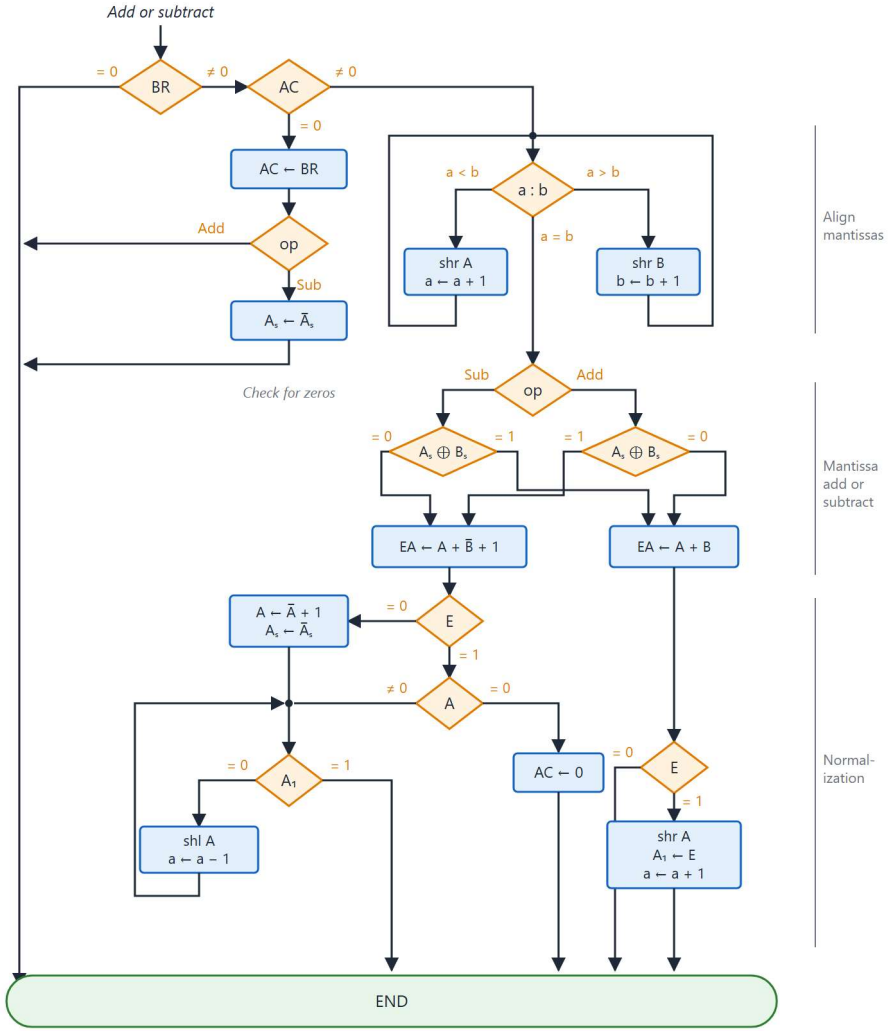
The registers

- **BR** (B_s, B, b): one operand: its sign, mantissa and biased exponent
- **AC** (A_s, A_1, A, a): the accumulator; A_1 is the MSB of the mantissa and must be 1 for a normalised number
- **QR** (Q_s, Q, q): the multiplier/quotient register in multiplication and division
- A **parallel adder** handles the mantissa arithmetic, the same circuit as fixed point
- A separate **parallel adder and comparator** handles the exponents

Addition and subtraction: four phases

- **1. Check for zeros:** if BR = 0 the result is already in AC; if AC = 0, AC ← BR (for subtraction the sign of AC is complemented)
- **2. Align mantissas:** shift the mantissa with the smaller exponent right, incrementing its exponent, until the exponents are equal
- **3. Add or subtract mantissas** by the signed-magnitude algorithm; if the carry E = 1 after addition, shift the sum right and increment the exponent
- **4. Normalise:** shift left and decrement the exponent until $A_1 = 1$; if the mantissa is all zeros, clear AC

Addition and subtraction of floating-point numbers



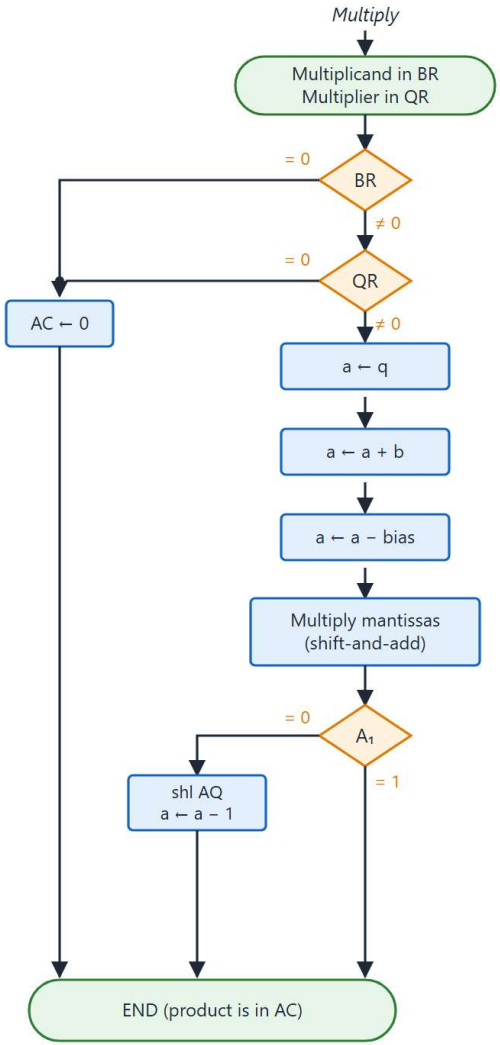
- Four phases: zero check, align, add or subtract, normalise
- Align: shift the mantissa with the smaller exponent right (left shifts would lose precision)
- Normalise: shift left and decrement the exponent until $A_1 = 1$
- A carry E = 1 after addition: shift right once, increment the exponent

Addition and subtraction of floating-point numbers (four phases). Adapted from M. Morris Mano, Computer System Architecture, Fig. 10-15.

Multiplication

- No mantissa alignment is needed
- **1. Check for zeros:** if either operand is zero, AC ← 0 and stop
- **2. Add exponents:** a ← q, then a ← a + b; the sum carries double the bias, so subtract the bias: a ← a − bias
- **3. Multiply mantissas** with the shift-and-add algorithm; the double-precision product is in AQ; only the single-precision part in AC is kept
- **4. Normalise:** if $A_1 = 0$, shift AQ left and decrement the exponent once (at most one leading zero is possible); if $A_1 = 1$ it is already normalised

Floating-point multiplication



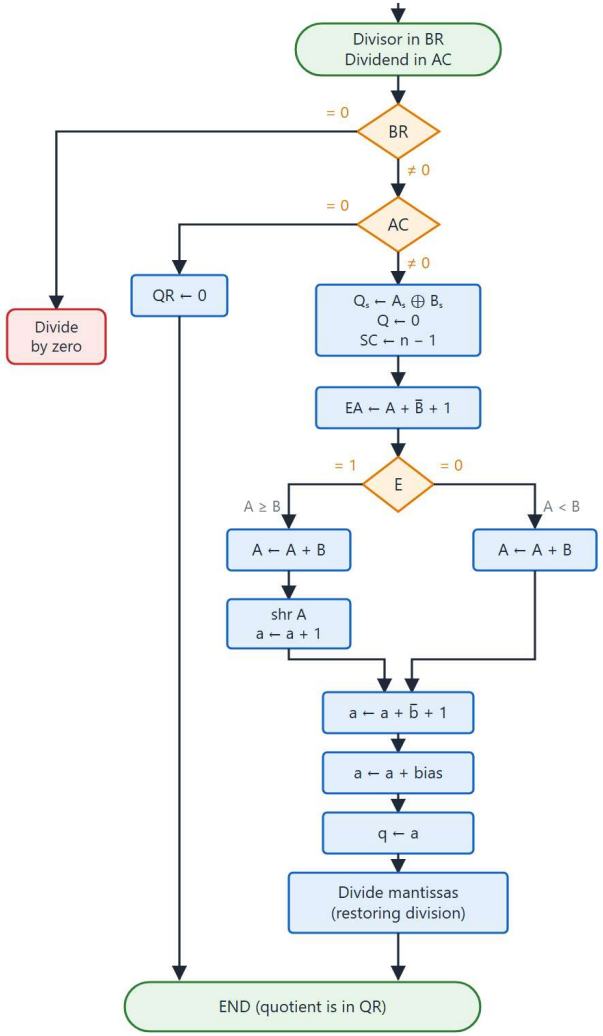
- Exponents: a + b = true a + true b + 2 × bias
- Subtracting one bias gives true a + true b + bias
- Normalise: at most one left shift of AQ (check A_1)
- No mantissa alignment is needed

Multiplication of floating-point numbers. Adapted from M. Morris Mano, Computer System Architecture, Fig. 10-16.

Division

- **1. Check for zeros:** BR = 0 is divide by zero, so terminate with an error; AC = 0 gives zero: QR ← 0 and stop
- **2. Initialise:** $Q_s \leftarrow A_s \oplus B_s$, Q ← 0, SC ← n − 1
- **3. Align the dividend:** EA ← A + B′ + 1; if E = 1 (A ≥ B), shift A right and increment a so that A < B; if E = 0, restore A. This prevents mantissa divide overflow
- **4. Subtract exponents:** a ← a + b′ + 1; add the bias back and transfer to q
- **5. Divide mantissas** by the division algorithm; since A < B the result is already normalised

Floating-point division



- Dividend alignment: if A ≥ B shift A right and increment a, so that A < B
- Then the mantissa quotient is a fraction below 1, already normalised
- Division is the only operation with no separate normalisation step
- Subtract the exponents, add the bias back and transfer to q

Division of floating-point numbers. Adapted from M. Morris Mano, Computer System Architecture, Fig. 10-17.

Summary: floating-point arithmetic operations

Operation	Exponent step	Mantissa step	Normalisation needed?
Add / Subtract	Compare; align smaller exponent	Add or subtract (signed-magnitude)	Yes: shift left until $A_1 = 1$, or shift right if overflow
Multiply	Add exponents; subtract bias	Multiply (shift-and-add); keep high half	At most one left shift (A_1 check)
Divide	Subtract exponents; add bias	Divide	No: dividend alignment guarantees a normalised result

Summary: floating-point arithmetic operations

Unit 4 summary (1 of 2)

- **4.1 Control memory:** a ROM of microinstructions; the CAR addresses it; one microprogram per machine instruction
- **4.2 Address sequencing:** increment, branch, map, call/return, chosen by the BR and CD fields
- **4.3 General registers:** SELA, SELB, SELD and OPR form a 14-bit control word for one shared ALU
- **4.4 Stack:** last-in, first-out with an SP; push and pop RTL; RPN is evaluated with a stack
- **4.5 Instruction formats:** opcode, address and mode fields; fewer address fields give shorter instructions but longer programs
- **4.6 Addressing modes:** nine rules for finding the operand through an effective address

Unit 4 summary (2 of 2)

- **4.7 Data transfer and manipulation:** transfer, arithmetic, logic and shift instruction groups
- **4.8 Program control:** branches, CALL/RET and interrupts, driven by the status bits C, S, Z and V
- **4.9 Addition and subtraction:** one adder for both; add or subtract magnitudes by the signs; V = $C_7 \oplus C_8$
- **4.10 Multiplication:** shift-and-add, Booth's algorithm for signed numbers, and the array multiplier
- **4.11 Division:** shift left, trial subtract, restore; DVF flags divide overflow
- **4.12 Floating point:** align exponents to add or subtract, add or subtract exponents to multiply or divide, then normalise
- **Key applications:** stacks and RPN in compilers; relative addressing for position-independent code; microprogramming so one processor can emulate another